EPSR-test-ex36 addato: inserting molecules into a box

Program(s): addato | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `addato` [1], which inserts copies of molecules at random positions and orientations into an existing `.ato` box without changing its size — used e.g. to add solutes to an equilibrated solvent before an EPSR run [2]. The case covers the command-line and stdin forms and the retry branches for non-existent files.

Code exercised

Fortran (original, fortran_src/):

- `addato.f`: `addato` (random insertion, template-name check, merging of molecule and atom types)
- `molfit.f`: `rotatm`; `auxiliary_routines.f`: `ran1`; `ato_manipulation_rev4.f`: `ftype`
- `ato_read_and_save.f`: `minitc` (read `.ato`), `moloutc` (write `.ato`), `normalise_coordinates`; `minimagec.f`: `minimagec`
- `file_and_directory_commands.f`: `search_for_files` (file prompt / directory listing), `get_full_filename`

Rust (port, rust/epsr-core/src/):

- `tools/addato.rs`: `run`, `ask_ato`; `tools/mixato.rs`: `SavedMol`
- `tools/setup_common.rs`: `rotatm`, `search_for_files`, `get_full_filename`
- `ato.rs`: `minitc`, `moloutc`, `ftype`, `normalise_coordinates`; `neighbour.rs`: `minimagec`; `rng.rs`: `Ran1`

Physical system

The EPSR-test configuration (200 dimethyl sulphoxide + 400 water molecules, 3200 atoms, 300 K, cubic box 33.1213 Å) [3] with 5 DMSO and 10 water molecules added (`bigbox.ato`, 615 molecules), then 4 more water (`bigbox2.ato`, 619 molecules); the box side stays 33.1213 Å, so the density rises. Positions and orientations come from the `ran1` generator state stored in the `.ato` file.

Inputs

`DMSO.ato`, `WaterMolecule.ato`, `200DMSO400Water.ato` from EPSR-test. `addato 2 < addato1.stdin`: add `DMSO.ato` (name with extension) ×5 and `WaterMolecule` ×10 to `200DMSO400Water`, output `bigbox`. `addato < addato2.stdin`: number of files asked (1), a non-existent file to add (`NoFile`, retried), `WaterMolecule` ×4, a non-existent target (`NoFile2`, retried), target `bigbox.ato`, output `bigbox2`.

How the accuracy was verified

`commands.txt` (2 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

Two program calls in one pass, the second reading the output of the first. Compared outputs: `bigbox.ato`, `bigbox2.ato`, `dirlist.txt`, all identical (header lines 615 and 619 molecules, box 0.33121300E+02 in the reference).

Notes

As in the Fortran, the template of every added molecule type becomes the name of the file it was read from: the existence test opens the blank-padded `character*256` template name, which never exists. Tethered molecules are not supported by the port's reader (not used here).

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).

[2] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).

[3] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).

[4] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.